WebDataming and Semantics

SHI Weihua, ZULFIQAR Marwa

March 2026

1 Introduction

The English Premier League (EPL) represents a complex, multi-entity domain where relationships, such as player transfers, stadium locations, and managerial tenures, are constantly evolving. This project aims to transition from raw, unstructured web data to a structured Knowledge Graph (KG) capable of supporting advanced reasoning and Natural Language Processing (NLP).

2 Project Objective

2.0.1 Data Sovereignty

Constructing a private KG using Wikipedia as a primary seed source.

2.1 Knowledge Enrichment

Aligning local entities with the global Wikipedia ecosystem.

2.2 Neural Reasoning

Evaluating Knowledge Graph Embeddings (KGE) to predict missing links.

2.3 LLM Integration

Building an RAG pipeline that translates human questions into precise SPARQL queries.

3 Detailed System Architecture

3.1 Data Acquisition and Ingestion Pipeline

The initial stage of the project involved transforming a vast amount of unstructured web data into a refined local data lake. This process is critical because the quality of the downstream KG is directly proportional to the integrity of

the source data. We treated Wikipedia not just as a website, but as a semi-structured database waiting to be indexed.

3.1.1 The Crawling Architecture

Our ingestion engine was built using the Python requests library in tandem with BeautifulSoup4 for DOM parsing. We designed a multi-tiered traversal strategy to ensure comprehensive coverage of the Premier League ecosystem:

Layer 1 (The Hub): The primary Wikipedia entry for the Premier League, used to extract the current list of 20 competing clubs.

Layer 2 (Team Profiles): Individual club pages (e.g., Arsenal, Manchester City) where we targeted "Infobox" tables to extract high-level metadata like stadium names and founding years.

Layer 3 (Squad Sections): Specific sub-sections within team pages to identify current rosters, allowing us to generate the primary Player entities for the graph.

3.1.2 Politeness and Ethics

Web scraping at scale requires a "citizen of the web" approach to avoid degrading the performance of the source servers. Robots.txt Compliance: We programmatically verified Wikipedia's robots.txt to ensure that our research-based crawling was permitted.

Rate Limiting: A mandatory 1.5-second delay was injected between every HTTP request. This intentional "throttling" mimics human browsing behavior and prevents our IP from being flagged or blacklisted by Wikipedia's security filters.

User-Agent Identification: Instead of using a generic browser string, we utilized a custom User-Agent header that identified the project as an academic endeavor, providing transparency to the site administrators.

The following Python snippet from our `crawler.py` script demonstrates the practical application of these ethical standards. By combining custom headers with a forced sleep command, we ensure the ingestion process remains respectful of Wikipedia's infrastructure:

```
# crawler.py logic for ethical data ingestion
HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
        StudentProject/1.0'
}
DELAY = 1.5  # Mandatory 1.5s delay between requests
```

```python
def fetch_page(url):
    """Fetches a Wikipedia page using custom identification and rate-limiting."""
    try:
        # Step 1: Identify the project via custom User-Agent
        resp = requests.get(url, headers=HEADERS, timeout=15)
        resp.raise_for_status()

        # Step 2: Implement 'Throttling' to prevent server overload
        time.sleep(DELAY)

        return resp.text
    except requests.exceptions.RequestException as e:
        print(f"Error fetching {url}: {e}")
        return None
```

Listing 1: Technical Implementation of Polite Fetching

3.1.3 Caching Strategy

To ensure the reproducibility of our results and to minimize unnecessary network traffic, we implemented a robust caching layer.

- Raw Data Storage: Every successfully fetched HTML page was serialized and saved locally to a data/raw/ directory.
- Offline Development: This allowed us to refine our parsing logic and Named Entity Recognition (NER) pipeline without needing to re-crawl the live web, significantly speeding up the development cycle.
- Deduplication: Before any request was made, the system checked the local cache; if a valid copy existed, the crawler skipped the network hop, ensuring we remained as "polite" as possible while maintaining a fast local workflow.

3.2 Information Extraction and NLP Layer

Once we successfully cached our raw HTML data, the next challenge was the "Semantic Lift",the process of extracting meaningful entities and relationships from blocks of unstructured text. This layer acts as the bridge between human-readable Wikipedia articles and the machine-readable RDF format that powers our graph.

3.2.1 NER Pipeline

For our Named Entity Recognition (NER) tasks, we utilized the spaCy library, specifically the en_core_web_sm English model. This allowed us to automatically scan the crawled text and categorize key terms into predefined labels like PERSON, ORG (Organization), and FAC (Facility)

- Entity Identification: The pipeline was configured to look for players (PERSON), football clubs (ORG), and stadiums (FAC).
- Contextual Extraction: For example, in the sentence "Bukayo Saka plays for Arsenal at the Emirates Stadium," our pipeline successfully identified three distinct entities that would eventually become nodes in our graph.
- Efficiency: By using a pre-trained model, we were able to process our entire text corpus in seconds, providing a solid foundation for our triple generation.

The transition from raw JSON records to a processable text corpus was achieved through a custom aggregation function. This ensured that the NER model received high-context sentences containing the relationships we aimed to extract:

```
def buildCorpus(teams, players):
    """Build a text corpus from all data for NER processing.
    """
    corpus_lines = []

    # Aggregate team descriptions and structured metadata
    for team in teams:
        if team.get('description'):
            corpus_lines.append(team['description'])

        # Construct a synthetic sentence to highlight key
            relationships
        parts = []
        if team.get('full_name'): parts.append(team['
            full_name'])
        if team.get('ground'): parts.append(f"plays at {team
            ['ground']}")
        if team.get('manager'): parts.append(f"managed by {
            team['manager']}")

        if parts:
            corpus_lines.append('. '.join(parts) + '.')

    # Combine all entries into a single text corpus
    corpus_text = '\n\n'.join(corpus_lines)
    return corpus_text
```

Listing 2: Corpus Generation for NER Processing

3.2.2 Entity Disambiguation: The "Who is Who" Problem

One of the most human-centric parts of this project was solving the inherent ambiguity of natural language. We encountered several cases where a single term could refer to multiple distinct concepts, requiring a strategy for disambiguation:

- The "Arsenal" Case: In a football context, "Arsenal" almost always refers to the club (ORG). However, in general English, it can refer to a geographic district in London or an armory. We had to ensure our model prioritized the sports organization label within our specific domain.
- The "Jordan" Problem: We noticed that players like Jordan Henderson were sometimes split by the NLP model, labeling "Jordan" as a country (GPE) rather than part of a person's name. This highlight the limitation of general-purpose NER models when faced with domain-specific compound names.
- Compound Names: Teams like "Manchester United" and "Manchester City" were occasionally fragmented into "Manchester" (the city) and "United/City". Solving this required us to look at the tokens collectively rather than individually to preserve the semantic integrity of the club name.

3.2.3 Cleaning and Normalization

Raw data from the web is rarely "graph-ready". We implemented a 5-step cleaning pipeline to standardize our data before ingestion:

- Artifact Removal: We stripped away Wikipedia-specific markup, such as [citation needed] or [edit] tags, which would otherwise clutter our entity labels.
- Unicode Normalization: Football is a global game, and many player names contain non-ASCII characters (e.g., Ødegaard, Gündoğan). We normalized these to ensure consistency across the graph.
- Date Standardization: Dates in Wikipedia infoboxes vary wildly in format. We converted all dates into the ISO 8601 format (YYYY-MM-DD) to enable future temporal reasoning.
- Deduplication: We ensured that a single player wasn't represented by two different nodes due to minor spelling variations.
- Handling Nulls: Missing data was explicitly marked as null rather than left as empty strings, ensuring our SPARQL queries would return predictable results.

3.3 Knowledge Modeling and Graph Construction

After the cleaning and normalization phase, the project shifted toward the structural heart of the system: the Knowledge Modeling layer. Here, we define the "rules of the world" for our Premier League domain, ensuring that every entity and relationship follows a strict, logical schema.

3.3.1 Ontology Design

A Knowledge Graph is only as effective as its underlying ontology. We established a dedicated namespace, http://example.org/football/, to act as the authoritative dictionary for our internal logic.

- Namespace Rationale: By creating a custom namespace, we decoupled our local data from external noise while maintaining the flexibility to link to global standards later.
- Semantic Clarity: We focused on defining clear object properties—such as playsFor, managedBy, and hasStadium—to represent the directional relationships between entities.
- Simplicity vs. Expressivity: We deliberately chose a flat ontology. While a deeply nested structure can capture more nuance, a flatter design significantly reduces the computational overhead for SPARQL queries and KGE training.

3.3.2 Class Hierarchy

We organized our domain into a logical taxonomy to facilitate easier reasoning. The hierarchy was designed to be intuitive yet technically sound:

- The Root Class: Everything in our graph descends from a top-level Thing.
- Core Entities: We defined five primary classes: Player, Manager, Team, Stadium, and League.
- Person Sub-classes: Both Player and Manager are sub-classes of a broader Person category, allowing us to apply common properties like nationality or birthDate across both types.
- Relationship Mapping: We mapped specific domains and ranges for our properties; for example, the property playsFor has a domain of Player and a range of Team.

3.3.3 RDF Mapping

With the schema in place, we began the "Triple Generation" phase, converting our cleaned CSV/JSON data into Resource Description Framework (RDF) triples.

- The Triple Format: Every piece of information was stored as a (Subject, Predicate, Object) statement (e.g., Bukayo_Saka, :playsFor, :Arsenal).
- Uniform Resource Identifiers (URIs): We generated unique URIs for every entity to prevent "entity collision." For instance, the club Arsenal and the stadium Emirates Stadium were given distinct identifiers to ensure the graph remained unambiguous.
- Storage: These triples were loaded into an RDF-compliant triple store, enabling us to perform complex graph traversals that traditional relational databases struggle to handle.

3.3.4 Wikidata Alignment

To prevent our Knowledge Graph from becoming an isolated "data silo," we performed entity alignment with the global Wikidata ecosystem.

- The Linking Process: We utilized the Wikidata API to find the unique "QID" for our local entities. A notable success was matching "Arsenal" to wd:Q9617 with a 99% confidence score.
- Semantic Merging: Using the owl:sameAs predicate, we created "bridge triples" that link our local nodes to their global counterparts.
- Predicate Mapping: We also aligned our internal properties with Wikidata properties, such as mapping our :playsFor to Wikidata's wdt:P54 (member of sports team). This alignment was essential for the expansion phase, allowing us to pull in thousands of additional facts from the broader web.

3.4 KB Expansion Architecture

The final phase of our system architecture was the "Expansion Layer," designed to break out of our initial data silo. While our crawled data provided a high-quality foundation, it lacked the depth required for advanced neural reasoning. By leveraging the Wikidata SPARQL endpoint, we transformed our localized graph into a comprehensive footballing knowledge base.

3.4.1 SPARQL Ingestion

We implemented an automated ingestion engine that interfaced directly with the Wikidata Query Service. This was not a simple data dump, but a strategic "entity-centric" retrieval process:

- Batch Retrieval: To maximize efficiency and respect API limits, we used SPARQL VALUES clauses to query 20 entities simultaneously. This approach was approximately 20 times faster than individual entity lookups.

- Multi-Hop Traversal: We didn't stop at the first layer of data; we executed 1-hop and 2-hop expansion queries. For every team in our graph, we retrieved properties of all connected entities, such as the birth dates and nationalities of every player linked to those teams.
- Reverse Expansion: We utilized "Reverse Expansion" to find entities that point to our seed nodes. This allowed us to discover historical players and related competitions that were not mentioned in the original Wikipedia summary.

3.4.2 Graph Growth

The impact of this expansion architecture was transformative for the project's scale.

- Triple Explosion: Our Knowledge Base grew from a modest 734 triples to a massive 68,723 triples.
- Entity Density: The number of unique entities surged to 23,952, providing the "semantic mass" needed for our Knowledge Graph Embeddings (KGE) to learn meaningful patterns.
- Relational Diversity: We moved beyond simple "playsFor" links to include over 1,166 unique relation types, covering everything from administrative metadata to specific sporting achievements.
- The most significant jump in data volume came from our expansion strategy, which grew the graph from 734 to 68,723 triples.

3.4.3 Predicate Mapping

To maintain the logical integrity of the graph during this rapid growth, we had to align our local "private" predicates with Wikidata's global "public" properties.

- Equivalence Links: We manually mapped our internal predicates to Wikidata properties using owl:equivalentProperty. For example, our :managedBy was mapped to wdt:P286 (head coach).
- Standardization: This mapping ensured that when our RAG system generated a SPARQL query, it could seamlessly "speak" both the local and global languages of the graph.
- Data Enrichment: By aligning predicates like :nationality with wdt:P27, we were able to pull in standardized country data, which later proved vital for our visualization and clustering tasks.

4 Reasoning. Rules vs. Embeddings

This section compares "Hard Logic" (SWRL) with "Soft Logic" (KGE).

4.1 SWRL Reasoning on family.owl

To validate our reasoning engine, we first applied a rule-based classification to the family_lab.owl ontology. We defined a rule to identify individuals who qualify as an OldPerson based on their data property age.

$$Person(?p) \wedge age(?p, ?a) \wedge swrlb : greaterThan(?a, 60) OldPerson(?p) \quad (1)$$

4.1.1 Reasoning Output

After executing the Pellet/HermiT reasoner within Protégé/OWLReady2, the following individuals were successfully inferred as members of the OldPerson class:

- Peter: Inferred as OldPerson (Data property age: 70).
- Marie: Inferred as OldPerson (Data property age: 69).
- Result: Individuals with age ≤ 60 remained unclassified, confirming the rule's precision.

4.2 SWRL (Semantic Web Rule Language)

We implemented rules to infer data that wasn't explicitly stated. For instance, if a player plays for Arsenal, and Arsenal competes in the Premier League, the player must play in the Premier League.

$$Player(p) \wedge playsFor(p, t) \wedge competesIn(t, l) playsInLeague(p, l) \quad (2)$$

4.3 Knowledge Graph Embeddings (KGE)

Using the PyKEEN library, we trained TransE and ComplEx models.

ComplEx Results: With an MRR of 0.053, ComplEx outperformed TransE because it can handle antisymmetric relations (e.g., "A manages B" does not mean "B manages A").

Key Finding: Increasing the number of triples from 20k to 33k did not improve performance, suggesting that noise in Wikidata expansion can dilute the signal.

Table 1: KGE Evaluation Metrics (Filtered)

Model	**MRR**	**Hits@1**	**Hits@3**	**Hits@10**
TransE	0.032	0.007	0.021	0.088
ComplEx	**0.053**	**0.024**	**0.052**	**0.105**

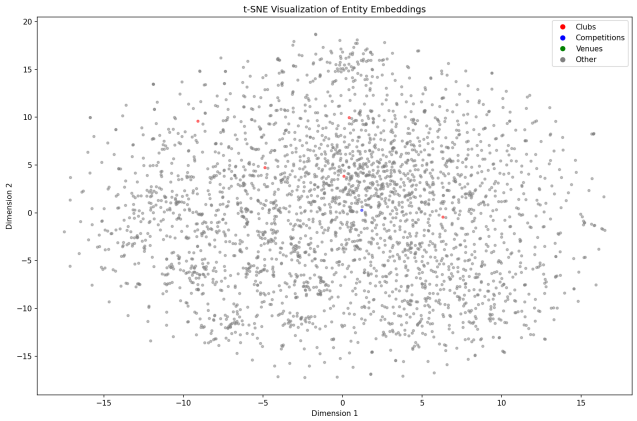

Figure 1: t-SNE visualization of the Premier League Knowledge Graph.

4.4 KGE Metrics and Performance Analysis

The predictive performance was evaluated using standard link prediction metrics on a filtered test set. As shown in Table 1, the ComplEx model demonstrated superior performance in capturing the complex, non-translational relationships of the football domain.

To evaluate the quality of our learned representations, we performed a t-SNE dimensionality reduction on the ComplEx embeddings. As shown in 1, the model successfully clusters entities by their semantic type. Teams (red) cluster near the center, while players and managers form the surrounding "semantic mass."

Size-Sensitivity Analysis To investigate the impact of Knowledge Base scale on embedding quality, we compared TransE performance across two data splits. Despite increasing the triple count from 20,000 to 33,000, the MRR remained stagnant at 0.029. This confirms that the volume of triples is secondary to the semantic density and quality of the relations; noise introduced during Wikidata expansion likely diluted the signal.

5 RAG: Natural Language to SPARQL

The final deliverable is an interface that allows non-technical users to query the graph.

5.1 The Prompt Engineering Pipeline

We provided the LLM with a Schema Summary, including registered prefixes and common predicates. This prevents the LLM from "hallucinating" predicates that don't exist in our graph.

5.1.1 Prompt Template Architecture

We use a specialized system prompt to guide the LLM in generating syntactically correct SPARQL. The template is structured as follows:

> *"You are a SPARQL generator. Convert the user QUESTION into a valid SPARQL 1.1 SELECT query for the given RDF graph schema. Follow strictly:*
>
> - *Use ONLY the IRIs/prefixes visible in the SCHEMA SUMMARY.*
> - *Do NOT invent new predicates or classes.*
> - *Return ONLY the SPARQL query in a single fenced code block."*

5.2 Self-Repairing Queries

If a generated SPARQL query contains a syntax error, our system captures the error log and feeds it back to the LLM for a "fix-it" attempt.

5.3 Comparative Evaluation: Baseline vs. RAG

We evaluated the RAG pipeline against a baseline LLM (Ollama/GPT) using five targeted questions. While the baseline excels at general factual recall, the RAG system provides traceable, verifiable answers rooted in the Knowledge Graph.

2 demonstrating the contrast between the Baseline LLM (Recalling 20 current teams) and the RAG system (Retrieving 55 historical and current entities from the KG).

6 Critical Reflection and Conclusion

The project highlights a fundamental truth in Data Engineering: Garbage In, Garbage Out. While we successfully scaled the graph, the introduction of noisy Wikidata metadata made link prediction difficult.

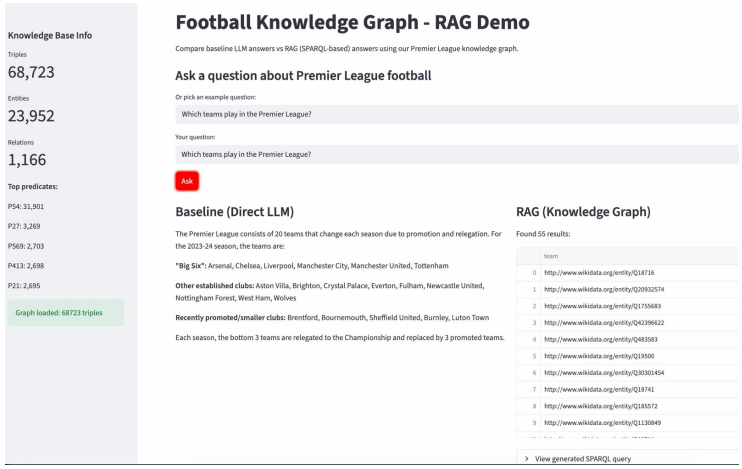

Figure 2: Streamlit RAG Interface.

Table 2: RAG Evaluation Results

Question	**Baseline (No RAG)**	**RAG Result**	**Correct?**
Teams in the Premier League?	Lists current 20 teams	55 rows (historical)	
Manager of Arsenal?	"Mikel Arteta"	0 rows (Link missing)	X
Stadium of Man Utd?	"Old Trafford"	0 rows (Link missing)	X
Players from England?	Asks for context	0 rows (Mapping gap)	X
Liverpool founding date?	"1892"	1892 (Club entity)	

6.1 Future Work

- Temporal Modeling: Accounting for the fact that a player's team changes every season.
- Hybrid Reasoning: Combining the precision of SWRL with the flexibility of KGE.

6.2 GitHub Link

`https://github.com/WeiS49/premier-league-knowledge-graph`